

SC6113 Individual Assignment

SafePay: A Financial Escrow Decentralized Application

Ethereum Sepolia · Solidity · MetaMask · Flask

Student Name

[REPLACE]

Matriculation Number

[REPLACE]

Course

SC6113

Submission Date

[REPLACE]

Submission note: highlighted placeholders must be replaced after the Sepolia deployment and live testing run.

1. Introduction

Decentralized applications (DApps) combine a user-facing application with blockchain-based logic. For financial use cases, smart contracts can enforce payment rules transparently without relying on a single centralized operator. This project implements SafePay, a peer-to-peer escrow DApp designed for a simple buyer-seller payment workflow on the Ethereum Sepolia test network.

2. Problem Statement

A common trust problem appears in online peer-to-peer transactions. A buyer may be unwilling to pay before goods or services are delivered, while a seller may be unwilling to deliver without confidence that the buyer has committed the funds. Traditional escrow services solve this problem through a trusted intermediary, but the intermediary controls the process and introduces additional dependency. SafePay replaces the basic holding function with a smart contract that temporarily locks ETH and releases it according to predefined rules.

3. Objectives

- Develop an original financial DApp that demonstrates a practical escrow use case.
- Implement the escrow rules in Solidity and deploy the contract to Ethereum Sepolia.
- Support MetaMask wallet connection and user-signed blockchain transactions.
- Provide a clear web interface for buyers and sellers.
- Integrate a Flask backend and SQLite activity log.
- Implement input validation, role-based authorization, transaction-state checks, and error handling.
- Evaluate the application using successful, invalid, and unauthorized transaction scenarios.

4. System Architecture

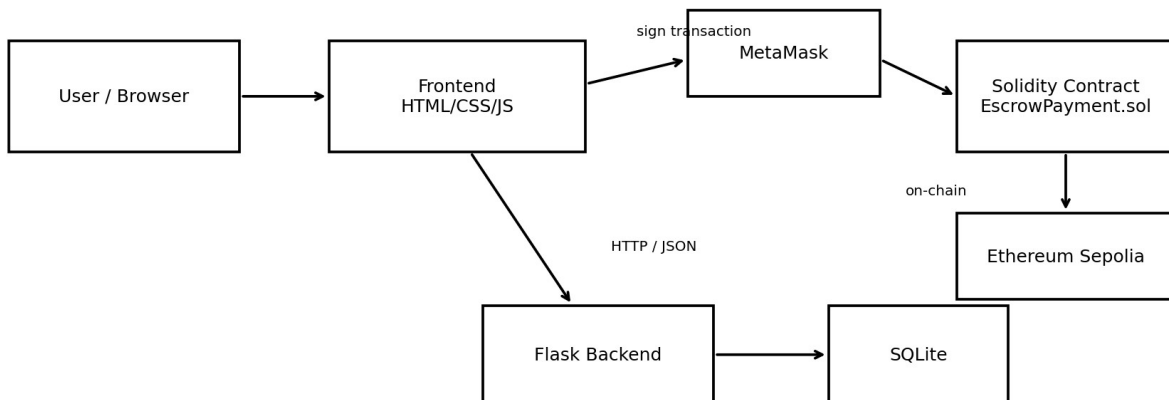


Figure 1. SafePay system architecture.

5. Technologies Used

Component	Technology	Purpose
Blockchain	Ethereum Sepolia	Test network for contract deployment and transactions
Smart Contract	Solidity 0.8.x	Implements escrow states, roles, and ETH transfer rules
Wallet	MetaMask	Account connection, private-key custody, and transaction signing
Web3 Library	ethers.js	Connects the browser to MetaMask and the smart contract
Frontend	HTML, CSS, JavaScript	User interface and transaction workflow
Backend	Flask (Python)	Serves configuration and records off-chain activity
Database	SQLite	Stores application activity records
Development / Deployment	Remix IDE	Compiles and deploys the Solidity contract

6. Smart Contract Design

The EscrowPayment contract supports multiple independent escrows. Each escrow stores the buyer address, seller address, locked amount, current status, and creation timestamp. The status model contains Funded, Delivered, Released, and Refunded states.

6.1 Core Functions

createEscrow(seller): Called by the buyer with ETH. Validates the seller address, buyer/seller separation, and positive payment amount.

markDelivered(id): Called only by the seller while the escrow is Funded. Changes the status to Delivered.

releaseFunds(id): Called only by the buyer after delivery is confirmed. Transfers the locked ETH to the seller.

refundBuyer(id): Allows the seller to return funds before release, changing the state to Refunded.

getEscrow(id): Returns the stored escrow information for the web interface.

6.2 Events and On-chain Records

The contract emits EscrowCreated, DeliveryConfirmed, FundsReleased, and Refunded events. These events provide an auditable record of the major state transitions. The DApp also shows Etherscan links so that the user can independently inspect confirmed blockchain transactions.

7. Application Design

The frontend is designed around a small number of visible actions. A wallet panel shows the connected account, network, and ETH balance. A create-escrow form accepts the seller address and ETH amount. The escrow table changes available actions according to the connected account role and current smart-contract state.

7.1 Buyer Workflow

1. Connect MetaMask on Sepolia.
2. Enter a seller address and payment amount.
3. Create and fund the escrow through a MetaMask transaction.
4. Wait until the seller confirms delivery.
5. Release the funds to the seller through a second MetaMask transaction.

7.2 Seller Workflow

1. Connect the seller MetaMask account.
2. Locate an escrow where the connected account is the seller.
3. Mark the escrow as Delivered after completing the transaction.
4. Optionally refund the buyer before funds are released.

8. Backend Integration

The Flask backend has a deliberately limited responsibility so that private keys remain entirely in MetaMask. The `/api/config` endpoint provides the deployed contract address and Sepolia configuration. The `/api/activity` endpoints record submitted, confirmed, and failed actions in SQLite. This separates on-chain financial state from off-chain application activity and demonstrates full-stack integration without moving transaction signing to the server.

[INSERT SCREENSHOT: DApp home page with connected MetaMask wallet]

9. Implementation

The DApp uses ethers.js in the browser to create a BrowserProvider from MetaMask. After the user authorizes account access, the application checks that the connected network has chain ID 11155111 (Sepolia) and constructs an EscrowPayment contract instance using the deployed contract address and ABI.

When the buyer creates an escrow, the frontend validates the seller address and ETH amount before calling createEscrow. MetaMask displays the transaction for user approval. After submission, the UI shows the transaction hash and waits for blockchain confirmation. The same pattern is used for delivery confirmation, fund release, and refund transactions.

The web page reloads escrow state from the contract and displays only records in which the connected wallet is the buyer or seller. Contextual action buttons reduce user error: for example, Release Funds is shown to the buyer only after the contract reports the Delivered state.

9.1 Input Validation and Error Handling

- Invalid Ethereum addresses are rejected before a transaction is requested.
- Zero or negative payment amounts are rejected.
- Buyer and seller must use different addresses.
- Wrong-network connections are detected and reported.
- MetaMask transaction rejection is shown as a user-readable error.
- Smart-contract reverts enforce authorization and state rules even if frontend checks are bypassed.
- The backend validates wallet-address and transaction-hash formats before writing activity records.

[INSERT SCREENSHOT: successful Create & Fund Escrow transaction + Etherscan confirmation]

10. Security Considerations

Authorization is enforced by the smart contract rather than only by the interface. onlyBuyer and onlySeller modifiers restrict role-specific actions. State transitions also prevent actions from being executed in an invalid order. For example, payment cannot be released before delivery is confirmed.

The release and refund functions update contract state before the external ETH transfer, following the checks-effects-interactions pattern. Private keys are never collected by the frontend or backend; MetaMask retains custody and signs every state-changing transaction. Because the project is a coursework prototype, it should not be treated as production-ready or professionally audited.

11. Testing and Results

Test	Scenario	Expected Result	Live Result
T01	Connect MetaMask on Sepolia	Address/network/balance shown	[RUN]
T02	Malformed seller address	Rejected before transaction	[RUN]
T03	Buyer = seller	Rejected	[RUN]
T04	Create escrow with 0.001 ETH	Confirmed; Funded state	[RUN]
T05	Buyer tries markDelivered	Contract reverts	[RUN]
T06	Seller marks delivery	Delivered state	[RUN]
T07	Seller tries releaseFunds	Contract reverts	[RUN]
T08	Buyer releases funds	Seller receives ETH; Released	[RUN]
T09	Reject MetaMask request	Readable rejection message	[RUN]
T10	Backend activity logging	Action appears in SQLite log	[RUN]

Live results are intentionally left as placeholders because they must be produced from the student-controlled MetaMask accounts and Sepolia deployment. Replace [RUN] with Pass/Fail and insert screenshots after completing the test run.

12. Challenges Encountered

The main implementation challenges are expected to occur at the boundary between the web interface and blockchain transactions: switching between buyer and seller accounts, ensuring MetaMask uses Sepolia, waiting for asynchronous transaction confirmation, and presenting smart-contract errors in a form that users can understand. The project structure isolates these responsibilities so that each component can be tested separately.

13. Limitations

- There is no independent dispute-resolution or arbitration mechanism.
- The prototype uses native ETH rather than stablecoins or other ERC-20 assets.
- The project is intended for Sepolia testnet rather than real-value production use.
- The frontend loads escrow records directly and would need event indexing for a large production dataset.
- The smart contract has not received a professional security audit.

14. Future Improvements

- Add ERC-20 or stablecoin escrow payments.
- Introduce an optional arbitrator for disputed transactions.
- Use an indexing service for scalable transaction history.
- Add notifications and richer transaction search/filtering.
- Add backend authentication for protected off-chain features.
- Conduct automated smart-contract unit tests and professional security review.

15. Conclusion

SafePay demonstrates an end-to-end financial DApp architecture using Solidity, MetaMask, Ethereum Sepolia, a browser-based frontend, Flask, and SQLite. The smart contract provides transparent role and state enforcement for a basic escrow workflow, while the web interface makes the blockchain interactions accessible to the user. After completing the live Sepolia deployment and test evidence, the project will satisfy the required demonstration of blockchain transactions, wallet integration, backend integration, validation, transaction history, testing, and security discussion.

References

1. Ethereum documentation - smart contracts and Sepolia test network.
2. Solidity documentation - language syntax, security considerations, and contract development.
3. MetaMask documentation - wallet connection and transaction signing.
4. ethers.js documentation - browser provider and contract interaction.
5. Flask documentation - backend routing and JSON APIs.